AlphaZero Capital

Master Reference — v2.0

Generated: Thu Mar 12 2026 | NSE India Autonomous Trading System

Project Scorecard

32	30	3	18	83
✅ Completed	🆕 Added in v2	🔶 Partial	❌ Still TODO	📋 Total Features

Key Files Added / Changed in v2

File	Location	What It Does
market_data.py	src/data/	yfinance fixes, NSE Bhav Copy, NSE Direct API, Screener.in scraper, live feed
cache.py	src/data/	SQLite + Parquet historical cache — data/cache/ohlcv/*.parquet, 7-day TTL for fundamentals
position_sizer.py	src/risk/	Full Kelly criterion + ATR sizing, PositionSizer class with trade history learning
correlation_control.py	src/risk/	60-day rolling correlation matrix — blocks r>0.8 pairs from entering portfolio
drawdown_breaker.py	src/risk/	7d/30d circuit breakers + PortfolioRebalancer — weekly drift correction
walk_forward.py	src/backtest/	Out-of-sample 6m-train/1m-test sliding windows — grades strategies A–D
order_manager.py	src/execution/	Order retry (3×), partial fill re-submission, bracket order (SL+target)
ops.py	src/infra/	Auto-restart systemd/PM2 config, 7-day log rotation, daily backup, /health data
backend.py	dashboard/	GET /health endpoint added, all imports safe, serves HTML at

File	Location	What It Does
		localhost:8000
main.py	root	Wires all new modules — drawdown, rebalance, walk-forward, order manager
llm_provider.py	src/agents/	Universal LLM: auto-detect Claude/GPT/Gemini/Ollama/LM Studio/HuggingFace
titan_agent.py	src/agents/	Removed broken numpy.iterable (NumPy ≥1.25 crash fix)

Section 1 — Complete Feature Status

Every feature tracked since v1. Status reflects current state after all patches applied.

✅ **DONE** = implemented and working 🆕 **NEW v2** = added in this patch 🔶 **PARTIAL** = exists but incomplete ❌ **TODO** = not yet built

Area	Feature	Status	Priority	Notes / Implementation Detail
Data Layer	**Multi-source price fetcher**	✅ DONE	—	OpenAlgo → yfinance → Stooq → Bhav Copy chain
Data Layer	**yfinance MultiIndex fix**	🆕 NEW v2	—	Flatten ('Open','TCS.NS') columns, retry on 429 rate-limit
Data Layer	**yfinance exponential backoff**	🆕 NEW v2	—	3 retries: wait 1s, 2s, 4s before giving up
Data Layer	**Historical data cache (Parquet)**	🆕 NEW v2	—	data/cache/ohlcv/*.parquet — never re-downloads stale data
Data Layer	**Cache TTL management**	🆕 NEW v2	—	1m/5m=4h, 1d=24h, fundamentals=7d, auto-cleanup at 30d
Data Layer	**NSE Bhav Copy (EOD)**	🆕 NEW v2	—	Full implementation — was a stub. Bulk CSV from nseindia.com
Data Layer	**NSE Direct API**	🆕 NEW v2	—	nseindia.com historical endpoint — no rate limits, faster
Data Layer	**Alpha Vantage integration**	✅ DONE	—	Accepts ALPHA_VANTAGE_KEY / ALPHAVANTAGE_API_KEY / ALPHA_VANTAGE_API_KEY
Data Layer	**Stooq EOD fallback**	✅ DONE	—	Free NSE prices, no API key, unlimited history
Data Layer	**Screener.in fundamentals**	🆕 NEW v2	—	Scrapes P/E, ROE, D/E, market cap — cached 7 days
Data Layer	**yfinance fundamentals fallback**	✅ DONE	—	yf.Ticker.info — fallback when Screener.in fails
Data Layer	**Candle pattern detection**	✅ DONE	—	8 patterns (Doji, Hammer, Engulfing etc) — pure pandas

Area	Feature	Status	Priority	Notes / Implementation Detail
Data Layer	**Volume analysis (OBV, ratio)**	✅ DONE	—	Accumulation/distribution, price-vol confirmation
Data Layer	**Live feed (background polling)**	🆕 NEW v2	—	start_live_feed() — background thread, 60s polling
Data Layer	OpenAlgo LIVE data	🔶 PARTIAL	HIGH	Works in MODE=LIVE. Needs broker API key in .env
Data Layer	Tickertape / Trendlyne feed	❌ TODO	LOW	Secondary fundamentals + analyst consensus
Risk/Capital	**CapitalAllocator — sigma weighted**	✅ DONE	—	Sigma score × sector cap, 5% per position max
Risk/Capital	**GUARDIAN risk rules**	✅ DONE	—	VIX scaling, consecutive loss limit, daily P&L gate
Risk/Capital	**Full Kelly criterion**	🆕 NEW v2	—	kelly_fraction() — win_rate × avg_win/loss. Half-Kelly default
Risk/Capital	**ATR position sizing**	✅ DONE	—	1% capital risk / 2×ATR stop distance
Risk/Capital	**Combined Kelly + ATR sizing**	🆕 NEW v2	—	optimal_position_size() — takes the more conservative of both
Risk/Capital	**Adaptive Kelly per strategy**	🆕 NEW v2	—	PositionSizer learns from trade history; updates per strategy
Risk/Capital	**Correlation control**	🆕 NEW v2	—	Blocks any stock with r>0.8 vs existing holdings (60d window)
Risk/Capital	**Portfolio rebalancer**	🆕 NEW v2	—	Weekly drift check — generates BUY/SELL to restore targets
Risk/Capital	**7-day rolling drawdown breaker**	🆕 NEW v2	—	Stops trading if 7d peak-to-trough exceeds DRAWDOWN_7D_LIMIT
Risk/Capital	**30-day rolling drawdown breaker**	🆕 NEW v2	—	Same for 30d window — DRAWDOWN_30D_LIMIT env var
Risk/Capital	**Daily P&L circuit breaker**	✅ DONE	—	MAX_DAILY_LOSS_PCT in .env — was already in GUARDIAN

Area	Feature	Status	Priority	Notes / Implementation Detail
Risk/Capital	**NAV history persistence**	🆕 NEW v2	—	logs/nav_history.json — daily NAV stored for drawdown calc
Risk/Capital	Monte Carlo stress test	❌ TODO	LOW	Simulate 1000 paths before deploying new strategy
Dashboard	**5-tab stock modal**	✅ DONE	—	Overview, Technical, Fundamental, Trade Setup, News
Dashboard	**Indicator tooltips on hover**	✅ DONE	—	RSI, ADX, EMA, ATR, MACD, VWAP — plain English
Dashboard	**Candle patterns display**	✅ DONE	—	Doji, Hammer, Engulfing, Morning/Evening Star, Inside Bar
Dashboard	**Volume analysis panel**	✅ DONE	—	OBV, vol ratio bar, accumulation/distribution label
Dashboard	**KARMA intelligence panel**	✅ DONE	—	Strategy weights, discovered patterns, regime accuracy
Dashboard	**MTF vote count in reason**	✅ DONE	—	e.g. '4/5 timeframe indicators aligned'
Dashboard	**Capital Deployed + Cash cards**	✅ DONE	—	Portfolio metrics strip, ₹ + % format
Dashboard	**STRONG_BUY badge colour fix**	✅ DONE	—	includes() check — no more red for STRONG_BUY
Dashboard	**Trade type + holding badge**	✅ DONE	—	INTRADAY/SWING/POSITIONAL/LONG TERM with colour codes
Dashboard	**GET /health endpoint**	🆕 NEW v2	—	Returns CPU%, RAM%, disk%, agent statuses as JSON
Dashboard	**Walk-forward results endpoint**	🆕 NEW v2	—	GET /api/walk-forward — OOS strategy grades
Dashboard	**Orders API endpoint**	🆕 NEW v2	—	GET /api/orders — live order state machine log
Dashboard	**CORS + file:// fix**	✅ DONE	—	HTML served at localhost:8000, not file://

Area	Feature	Status	Priority	Notes / Implementation Detail
Dashboard	TradingView live chart	❌ TODO	MEDIUM	Embed TradingView widget in Technical tab
Dashboard	Portfolio equity curve chart	❌ TODO	MEDIUM	Chart.js — capital growth + drawdown overlay
Dashboard	Telegram command log panel	❌ TODO	LOW	Last 10 commands + system replies in sidebar
Execution	**Paper executor (simulation)**	✅ DONE	—	Realistic slippage 0–16 bps, 100% fill rate
Execution	**OpenAlgo LIVE executor**	✅ DONE	—	Place/modify/cancel, mass cancel kill switch
Execution	**Telegram alerts**	✅ DONE	—	Trade alerts, stop hits, daily P&L briefing
Execution	**Telegram /pause /resume /kill**	✅ DONE	—	Guardian commands wired and tested
Execution	Telegram /status	🔶 PARTIAL	HIGH	Basic response exists; full agent health pending
Execution	**Order retry logic (3× backoff)**	🆕 NEW v2	—	500ms → 1s → 2s exponential backoff on rejected orders
Execution	**Partial fill handling**	🆕 NEW v2	—	Auto re-submits residual qty when broker fills partially
Execution	**Bracket orders (BO/CO)**	🆕 NEW v2	—	place_bracket_order() — SL+target in one call via OpenAlgo
Execution	**Order state machine**	🆕 NEW v2	—	PENDING→SUBMITTED→PARTIAL/FILLED/REJECTED, persisted to JSON
Learning	**KARMA real-time weight update**	✅ DONE	—	Every closed trade updates strategy weights
Learning	**Off-hours training (6PM–9AM)**	✅ DONE	—	Runs on last-fetched data across 5 timeframes
Learning	**Pattern discovery & publish**	✅ DONE	—	70%+ win rate in same setup → pattern logged
Learning	**LENS SQLite**	✅ DONE	—	WIN/LOSS/SCRATCH per signal with

Area	Feature	Status	Priority	Notes / Implementation Detail
	evaluation DB			timestamps
Learning	**Walk-forward backtesting**	🆕 **NEW v2**	—	6-month train / 1-month OOS sliding windows — grades A–D
Learning	**Strategy ranking by Sharpe**	🆕 **NEW v2**	—	walk_forward_results.json — ranked by avg OOS Sharpe
Learning	NEXUS XGBoost regime model	🔶 **PARTIAL**	**HIGH**	Stub + loader present. Needs 6-month training data collection
Learning	FinBERT news fine-tune	❌ **TODO**	**MEDIUM**	Fine-tune on Indian financial headlines (ET, MC)
Learning	RL PPO via Stable-Baselines3	❌ **TODO**	**LOW**	Full RL agent — Phase 3. Needs replay buffer infrastructure
Global Intel	Global news aggregator	❌ **TODO**	**HIGH**	RSS + NewsAPI: Reuters, Bloomberg, BBC, Al Jazeera — 15-min polling
Global Intel	India news monitor	❌ **TODO**	**HIGH**	ET, Moneycontrol, Business Standard, Livemint — domestic macro + policy
Global Intel	Geopolitical event classifier	❌ **TODO**	**HIGH**	LLM classifies news: WAR / SANCTIONS / TRADE / POLICY / DISASTER / ELECTION
Global Intel	Event → sector impact mapper	❌ **TODO**	**HIGH**	Rules engine: Oil War → BUY Energy, SELL Aviation; Fed Hike → SELL IT etc.
Global Intel	Commodity price tracker	❌ **TODO**	**HIGH**	Live: Crude Oil (Brent/WTI), Gold, Natural Gas, USD/INR, US 10Y yield
Global Intel	Macro event calendar	❌ **TODO**	**MEDIUM**	RBI MPC, US Fed, GDP, CPI release dates → pre-event signal dampening
Global Intel	Geopolitical risk score (0-100)	❌ **TODO**	**MEDIUM**	Composite risk index updated hourly — GUARDIAN scales position sizes down
Global Intel	Sector sensitivity matrix	❌ **TODO**	**MEDIUM**	Pre-built: which NSE sector moves how much per event type (historical)
Global Intel	Breaking news kill switch	❌ **TODO**	**HIGH**	GUARDIAN halts new orders when geo-risk > 80 or major event detected

Area	Feature	Status	Priority	Notes / Implementation Detail
Global Intel	News impact learning	❌ TODO	LOW	KARMA records which event types actually moved which stocks → refines rules
Infra/Ops	**GitHub repo structure**	✅ DONE	—	v2.0.1 branch, PATCH_GUIDE.md deployment map
Infra/Ops	**.env secrets management**	✅ DONE	—	All API keys, MODE, risk params in .env only
Infra/Ops	**All __init__.py files**	✅ DONE	—	10 packages — import resolution fixed
Infra/Ops	**Auto-restart (systemd + PM2)**	🆕 NEW v2	—	alphazero.service + ecosystem.config.js generated on first run
Infra/Ops	**Log rotation (7-day)**	🆕 NEW v2	—	Compresses to .log.gz after 7 days, deletes after 30
Infra/Ops	**Daily DB backup**	🆕 NEW v2	—	logs/evaluation.db + 7 other files → backups/YYYY-MM-DD/
Infra/Ops	**30-day backup pruning**	🆕 NEW v2	—	Old backup folders auto-deleted after 30 days
Infra/Ops	Cloud deployment (AWS/GCP)	❌ TODO	MEDIUM	EC2 t3.medium or GCP e2 — IST timezone critical
Infra/Ops	**Stale cache cleanup**	🆕 NEW v2	—	clear_stale_cache(30) runs after market close
Infra/Ops	**Windows setup.bat**	✅ DONE	—	One-click venv + deps + .env creation on Windows

Section 2 — What Is Still Incomplete

🔶 Partial — Needs Finishing

These exist in the codebase but are not fully functional:

Telegram /status command [HIGH]

Only basic response. Needs: open positions list, today P&L, GUARDIAN circuit breaker status, active agent count. Wire to dashboard/backend.py GET /api/status response.

OpenAlgo LIVE data [HIGH]

Works structurally. You must: (1) set OPENALGO_API_KEY in .env, (2) run OpenAlgo server locally, (3) connect your Zerodha/Upstox broker inside OpenAlgo, (4) set MODE=LIVE.

NEXUS XGBoost model [HIGH]

JSON model loader exists. Needs: 6 months of labeled NSE data (ADX, VIX, ATR → regime label), sklearn/xgboost training script, save to models/nexus_regime.json.

❌ TODO — Not Yet Built

These are confirmed gaps — the code path is missing or empty:

Monte Carlo stress test [LOW]

src/backtest/monte_carlo.py — simulate 1000 portfolio paths. Fire before any strategy goes LIVE. Use historical returns from LENS DB as input.

FinBERT fine-tune [MEDIUM]

src/agents/hermes_trainer.py — collect 10k Indian financial headlines from ET/Moneycontrol. Fine-tune bert-base on bullish/bearish/neutral labels.

RL PPO agent [LOW]

src/agents/karma_rl.py — replace KARMA rule weights with Stable-Baselines3 PPO. Needs gymnasium environment wrapper for the trade simulator.

TradingView chart embed [MEDIUM]

dashboard/alphazero-ui/src/components/TradingViewWidget.jsx — use TradingView Lightweight Charts (free, open source). One script import.

Portfolio equity curve [MEDIUM]

React component using recharts LineChart — read /api/portfolio history, plot daily NAV from nav_history.json.

Cloud deployment [MEDIUM]

AWS EC2 t3.medium (₹3k/month) or GCP e2-standard-2. Critical: set TZ=Asia/Kolkata. Use alphazero.service (already generated) for auto-start.

Tickertape / Trendlyne [LOW]

Scrape analyst buy/hold/sell ratings. Weight signals: if >70% analysts 'BUY', add 0.1 to sigma score.

Telegram command log [LOW]

Store last 10 (command, timestamp, result) in logs/telegram_log.json. Render in dashboard sidebar.

Section 3 — Suggested Enhancements

These go beyond the original spec. Ranked by trading impact.

Category	Idea	Impact	Effort	How to Implement
Data	**NSEpy direct integration**	HIGH	3 days	Replace yfinance for intraday — no rate limits, real-time
Data	**Redis OHLCV pub-sub**	HIGH	1 week	Agents subscribe to live ticks instead of polling — < 1s latency
Data	**Tickertape analyst consensus**	MEDIUM	2 days	Scrape buy/hold/sell ratings + price targets for signal weighting
Data	**Corporate actions calendar**	HIGH	3 days	Auto-detect ex-dividend, split, bonus dates → avoid holding through
Data	**F&O option chain live feed**	HIGH	1 week	IV, PCR, max pain from NSE option chain — regime confirmation
Risk	**Monte Carlo stress test**	HIGH	4 days	1000-path simulation before deploying any new strategy LIVE
Risk	**VaR (Value at Risk) daily report**	MEDIUM	2 days	95th percentile 1-day loss estimate — report in Telegram PM
Risk	**Sector concentration cap**	MEDIUM	1 day	Max 20% capital in any single sector (ATLAS already tracks this)
Risk	**Black-Scholes IV surface**	LOW	1 week	Model implied volatility for options-based hedging
AI/ML	**NEXUS XGBoost training pipeline**	HIGH	1 week	Auto-collect ADX/VIX/ATR labels → train → save nexus_regime.json
AI/ML	**FinBERT fine-tune on Indian news**	HIGH	2 weeks	ET, MC, Business Standard headlines → +15% sentiment accuracy
AI/ML	**Transformer price prediction**	MEDIUM	3 weeks	TFT (Temporal Fusion Transformer) on 1-min NSE candles
AI/ML	**Earnings surprise predictor**	MEDIUM	2 weeks	LLM on transcripts + past EPS trend → pre-earnings signal
AI/ML	**RL PPO agent**	LOW	1	Stable-Baselines3 PPO — full replacement for rule strategies

Category	Idea	Impact	Effort	How to Implement
	(Phase 3)		month	
AI/ML	**Regime clustering (k-means)**	MEDIUM	3 days	Unsupervised regime detection to complement NEXUS rules
Dashboard	**TradingView live chart embed**	HIGH	1 day	Lightweight widget — real candlestick chart in Technical tab
Dashboard	**Portfolio equity curve chart**	HIGH	2 days	Recharts — daily NAV line + drawdown shading from nav_history.json
Dashboard	**Heatmap of sector returns**	MEDIUM	2 days	D3.js treemap — visualise sector P&L across the watchlist
Dashboard	**Telegram command log**	LOW	1 day	Sidebar panel showing last 10 commands + system replies
Dashboard	**Mobile-responsive layout**	MEDIUM	3 days	React + Tailwind — monitor system from phone during market hours
Execution	**Zerodha Kite direct integration**	HIGH	1 week	kiteconnect Python SDK — bypass OpenAlgo for lower latency
Execution	**Smart order routing**	HIGH	1 week	Split large orders across time to minimise market impact
Execution	**OCO orders (One-Cancels-Other)**	MEDIUM	3 days	Pair SL and target — whichever triggers cancels the other
Execution	**Post-market GTT orders**	MEDIUM	2 days	Set Good-Till-Triggered orders overnight for next-day entries
Infra	**AWS/GCP cloud deployment**	HIGH	2 days	EC2 t3.medium — IST timezone, 99.9% uptime, static IP for broker
Infra	**Google Drive backup**	MEDIUM	1 day	gdrive API — nightly backup of evaluation.db + cache to Drive
Infra	**Grafana + Prometheus metrics**	MEDIUM	3 days	Time-series dashboarding — CPU/RAM/trade metrics over time
Infra	**Docker containerisation**	MEDIUM	2 days	docker-compose — reproducible environment, easy cloud deploy

Category	Idea	Impact	Effort	How to Implement
Reporting	**Weekly PDF performance report**	MEDIUM	3 days	ReportLab — strategy grades, equity curve, top/bottom trades
Reporting	**Daily Telegram summary**	HIGH	1 day	7PM message: trades today, P&L, win rate, best signal
Reporting	**Trade journal export (CSV/Excel)**	LOW	1 day	All closed trades → XLSX for broker reconciliation
Global Intel	**Global news aggregator agent**	HIGH	1 week	New HERMES_GLOBAL agent — polls Reuters/Bloomberg RSS every 15 min. LLM classifies each headline by event type + affected sectors.
Global Intel	**Event → sector impact engine**	HIGH	3 days	Hardcoded rules + learned weights. E.g. 'Middle East conflict' → Oil +8%, Aviation -5%, Defence +6%. Publishes SECTOR_SHOCK event.
Global Intel	**Commodity price feed**	HIGH	2 days	Yahoo Finance: BZ=F (Brent Crude), GC=F (Gold), NG=F (Nat Gas), USDINR=X. Poll every 5 min during market hours.
Global Intel	**Macro calendar integration**	HIGH	2 days	investing.com/economic-calendar scraper — auto-flag RBI MPC, US Fed, India CPI dates. Tighten stops ±2 days around events.
Global Intel	**Geopolitical risk index**	MEDIUM	1 week	0–100 composite score from: active conflict count, sanctions list changes, FII flow, VIX India. Scale position sizes inversely.
Global Intel	**NSE FII/DII flow monitor**	HIGH	2 days	NSE publishes daily FII buy/sell CSV. Scrape it. If FII net sell > ₹2000Cr 3 days running → reduce all positions 30%.
Global Intel	**Sector sensitivity matrix**	MEDIUM	3 days	Pre-built lookup: e.g. Energy stocks have 0.8 beta to crude oil. If crude up 5% → ONGC/Reliance expected +4%. Refine with KARMA.
Global Intel	**Breaking news kill switch**	HIGH	1 day	If LLM detects BREAKING: WAR / NUCLEAR / MARKET_HALT keyword → GUARDIAN pauses all new orders, Telegram alert sent immediately.
Global Intel	**US market pre-open scan**	MEDIUM	2 days	Run at 7:30 AM IST — check S&P 500 futures, NASDAQ futures, USD index. If futures down >1.5%, reduce intraday size by 50%.
Global Intel	**India VIX spike detector**	HIGH	1 day	India VIX from NSE. If VIX > 20: halve position sizes. If VIX > 25: paper-only mode. Already in GUARDIAN, needs VIX live feed.

Section 4 — Recommended Roadmap to LIVE Trading

Phase 1 — Foundation (Weeks 1–2): Paper Mode Only

Get the system running stably before risking any capital.

- Run python main.py and confirm all 16 agents start without errors
- Fill in .env — at minimum: ALPHA_VANTAGE_KEY, MODE=PAPER
- Watch logs/alphazero.log for 2 consecutive trading days — note any crashes
- Check data/cache/ohlcv/ is growing — confirms historical cache is working
- Review logs/backtest_results.json after first market close — confirm 5 strategies ran
- Set up auto-restart: copy alphazero.service to systemd (Linux) or run pm2 start ecosystem.config.js

Phase 2 — Validation (Weeks 3–8): Paper Mode + Monitoring

Build confidence in system behaviour before touching real money.

- Monitor dashboard at localhost:8000 daily — check agent statuses, circuit breaker state
- Review logs/walk_forward_results.json every Friday — only keep strategies graded A or B
- Complete Telegram /status command — you need real-time P&L visibility before going LIVE
- Train NEXUS XGBoost model — 6 weeks of data collection needed before this is useful
- Check win_rate in logs/tracker.json after each week — target > 55% before proceeding
- Verify backups are running: check backups/ folder has today's date directory by 11PM

Phase 3 — Go LIVE (Month 3): Real Capital, Small Size

Only proceed if: win_rate > 55%, Sharpe > 1.0, max_drawdown < 8% over 6 weeks of paper.

- Set MODE=LIVE in .env
- Connect OpenAlgo to your broker (Zerodha/Upstox) and set OPENALGO_API_KEY
- Start with ₹50,000 capital only — INITIAL_CAPITAL=50000
- Monitor EVERY trade for the first week manually
- Keep DRAWDOWN_7D_LIMIT=3.0 (tighter than default 5%) for the first month
- Scale: ₹1L after week 2, ₹2.5L after month 2, ₹5L+ after month 3

Phase 4 — Scale (Month 4+): Automation + Cloud

- Deploy to AWS EC2 t3.medium — IST timezone, static IP, 99.9% uptime
- Add Google Drive backup for evaluation.db
- Implement weekly PDF performance report via ReportLab

- Begin NEXUS XGBoost training if 6 months of labeled data collected
- Consider FinBERT fine-tuning if news sentiment accuracy feels unreliable

---

File locations: All new files are in alphazero_v2_complete.tar.gz. Extract and copy az_v2/* into your repo root. **Run: python patch_apply.py --apply --backup then python main.py**

Section 5 — Global & Regional News Intelligence

Markets are not just charts. A war breaking out in the Middle East, a US Fed rate decision, or India's CPI reading can move entire sectors more than any technical signal. This section defines how to wire global macro awareness directly into the trading engine.

Why This Matters — Real Examples

Event	Sector Affected	Direction	NSE Stocks Impacted	Typical Move
Middle East war / oil disruption	**Energy & Refining**	⬆️ **BUY**	ONGC, Reliance, BPCL, IOC	**+5% to +12%**
Middle East war / oil disruption	**Aviation**	⬇️ **SELL**	IndiGo (INDIGO), SpiceJet	**-4% to -10%**
Middle East war / oil disruption	**Paints / Chemicals**	⬇️ **SELL**	Asian Paints, Berger, Tata Chem	**-2% to -6%**
US Fed rate hike	**IT / Tech exports**	⬇️ **SELL**	TCS, Infosys, Wipro, HCL Tech	**-2% to -5%**
US Fed rate hike	**NBFC / Housing Finance**	⬇️ **SELL**	HDFC, Bajaj Finance, LIC Housing	**-3% to -7%**
US Fed rate cut	**IT / Tech exports**	⬆️ **BUY**	TCS, Infosys, Wipro, HCL Tech	**+2% to +5%**
Russia-Ukraine conflict	**Fertilisers / Agri**	⬆️ **BUY**	Chambal Fertilisers, Coromandel	**+6% to +15%**
Russia-Ukraine conflict	**Metals (aluminium)**	⬆️ **BUY**	Hindalco, Vedanta, NALCO	**+4% to +9%**
China slowdown / trade war	**Metals & Mining**	⬇️ **SELL**	Tata Steel, JSW Steel, SAIL	**-3% to -8%**
India-China border tension	**Defence**	⬆️ **BUY**	HAL, BEL, BEML, Paras Defence	**+5% to +20%**
India-China border tension	**Chinese-dependent IT**	⬇️ **SELL**	Dixon Tech, Amber Enterprises	**-4% to -10%**
RBI rate hike (India)	**Banks / NBFCs**	⬇️ **SELL**	HDFC Bank, ICICI, SBI, Axis	**-1% to -4%**
RBI rate cut (India)	**Real Estate**	⬆️ **BUY**	DLF, Godrej Properties, Sobha	**+3% to +8%**

Event	Sector Affected	Direction	NSE Stocks Impacted	Typical Move
INR depreciation (USD/INR up)	**IT exporters**	⬆️ **BUY**	TCS, Infy, Wipro (earn in USD)	**+1% to +3%**
INR depreciation (USD/INR up)	**Oil importers**	⬇️ **SELL**	BPCL, IOC (import crude in USD)	**-2% to -5%**
Gold price spike	**Jewellery / Gold ETF**	⬆️ **BUY**	Titan, Kalyan Jewellers, Muthoot	**+3% to +8%**
India budget — infra spend	**Infrastructure**	⬆️ **BUY**	L&T, IRB Infra, NCC, KNR Constructions	**+5% to +15%**
US recession fear	**IT / pharma (defensive)**	**MIXED**	IT down, Pharma defensive (Sun, Dr Reddy)	**+/- 3%**
Global pandemic / health crisis	**Pharma / Hospitals**	⬆️ **BUY**	Sun Pharma, Dr Reddy, Apollo Hospitals	**+8% to +30%**
El Nino / drought forecast	**FMCG / Consumer**	⬇️ **SELL**	HUL, ITC, Nestle (rural demand drop)	**-2% to -5%**

Architecture — How to Build It

The global news intelligence system plugs into the existing EventBus as a new agent: HERMES_GLOBAL. It sits alongside the existing HERMES (domestic news) agent and publishes GLOBAL_EVENT and SECTOR_SHOCK events that CHIEF, GUARDIAN, and TITAN already know how to consume.

New Agent: HERMES_GLOBAL (src/agents/hermes_global.py)

- Polls RSS feeds every 15 minutes: Reuters World, Bloomberg Markets, BBC Business, Al Jazeera Economy, NDTV Business, Economic Times
- Passes each headline through LLM (Claude/GPT): 'Classify this headline as one of: OIL_SHOCK, WAR, SANCTIONS, FED_DECISION, RBI_DECISION, TRADE_WAR, NATURAL_DISASTER, ELECTION, MARKET_HALT, OTHER. Also list affected NSE sectors: UP or DOWN.'
- Publishes GLOBAL_EVENT event to EventBus with: {type, headline, sectors_up: [], sectors_down: [], confidence: 0-1, source, timestamp}
- Deduplicated: same story from multiple sources counted once (hash of first 60 chars)
- Rate limit guard: max 5 LLM calls per minute to stay within API limits

New Module: Event → Sector Mapper (src/data/geo_impact.py)

- Hard-coded impact matrix (based on the table above) as a starting point
- KARMA learns and refines: after each event, compare predicted vs actual sector move → update weights
- Returns: {symbol: expected_move_pct} for each affected NSE stock in watchlist
- TITAN uses this as an additional signal layer on top of technical indicators

Commodity Price Feed (added to market_data.py)

- Brent Crude: yfinance BZ=F — poll every 5 min during market hours
- Gold: GC=F — correlates with INR weakness, safe-haven flows
- Nat Gas: NG=F — affects GAIL, IGL, MGL, Indraprastha Gas
- USD/INR: USDINR=X — critical for IT exporters and oil importers
- US 10Y Yield: ^TNX — signals risk-on/off globally; affects FII flows into India
- India VIX: INDIAVIX.NS — already in GUARDIAN, ensure live feed is wired

GUARDIAN Integration — Breaking News Kill Switch

- If geo_risk_score > 80: pause all NEW orders, keep existing positions with tightened stops
- If event type = MARKET_HALT or NUCLEAR or PANDEMIC_LEVEL_3: emergency stop, flatten all positions
- Telegram alert fired immediately regardless of time of day
- Geo risk score = weighted sum of: active_conflicts(×30) + sanctions_active(×20) + vix_india(×25) + fii_net_sell_streak(×25)

Macro Calendar Integration (src/data/macro_calendar.py)

- investing.com/economic-calendar — scrape weekly, store in data/cache/macro_events.json
- Key events to track: RBI MPC (8 per year), US FOMC (8 per year), India CPI (monthly), India GDP (quarterly), US NFP (monthly)
- Pre-event: 2 days before major event → halve position sizes, widen stops
- Post-event: if surprise > 0.5 std dev from forecast → TITAN gets 'MACRO_SURPRISE' signal boost

Data Sources — Free vs Paid

Source	What It Covers	Cost	Rate Limit	How to Use
Reuters RSS	Global news, geopolitics	Free	Unlimited	feedparser library — feed URL: feeds.reuters.com/reuters/worldnews
NewsAPI.org	1000s of news sources	Free (100/day)	100 req/day	newsapi-python library. NEWSAPI_KEY in .env
Yahoo Finance (yf)	Commodity prices, FX, VIX	Free	~2000 req/day	Already integrated. Add BZ=F, GC=F, USDINR=X to polling list
investing.com	Macro calendar, events	Free	Scraping OK	BeautifulSoup scraper — already have BS4 installed
NSE India (nse.gov)	FII/DII flows, India VIX	Free	Unlimited	Already have NSE direct session. Add /api/fiidiiTradeReact endpoint
GDELT Project	Global events database	Free	Unlimited	gdelt-doc API — real-time geopolitical event stream, no key needed

Source	What It Covers	Cost	Rate Limit	How to Use
Alpha Vantage NEWS	Financial news + sentiment	Free (25/day)	25 req/day	Already have AV key. Add NEWS_SENTIMENT function call
Telegram bot (push)	Breaking news Telegram alerts	Free	30 msg/sec	Already integrated — add GLOBAL_EVENT → Telegram notification

Implementation Priority

- Step 1 (1 day) — Commodity price polling: add BZ=F, GC=F, USDINR=X, ^TNX to market_data.py's polling list. Zero new infrastructure needed — already have yfinance.
- Step 2 (2 days) — FII/DII flow scraper: NSE publishes this daily. If FII net sell for 3 consecutive days > ₹2000Cr total → reduce position sizes 30%. Simple scraper, huge impact.
- Step 3 (3 days) — RSS news aggregator: feedparser + simple keyword rules (no LLM needed initially). Keywords: 'oil', 'crude', 'war', 'sanction', 'rate hike', 'inflation'. Map to sectors via geo_impact.py.
- Step 4 (1 week) — LLM classification layer: replace keyword rules with Claude/GPT call. More accurate, handles nuanced headlines. Reuse existing LLMProvider from src/agents/llm_provider.py.
- Step 5 (2 days) — GUARDIAN integration: geo risk score → position size scaling. India VIX live feed → GUARDIAN already checks it, just needs the live number.
- Step 6 (ongoing) — KARMA learning: after each geo event, log which stocks actually moved and by how much. Feed back into geo_impact.py weights. System gets smarter over time.

Key insight: Most retail traders and algos react AFTER the price moves. This system should detect the news FIRST and pre-position or exit BEFORE the crowd. Even a 5-minute head start on a major event is worth more than all the RSI/MACD signals combined.

Section 6 — Scanning All 5000 NSE Stocks in Under 10 Seconds

The naive approach — loop over 5000 symbols, fetch each one, run indicators — takes 8+ hours. The right approach uses a 3-stage funnel that eliminates 98% of stocks before running any heavy computation, combined with one bulk HTTP request, parallel async I/O, and vectorised numpy math on pre-cached data. Total time: under 10 seconds.

Why Naive Is Impossible — The Math

Approach	Time Per Stock	Total for 5000	Fatal Problem
Serial yfinance API calls	0.5 – 2s	**~83 min**	Each HTTP call blocks the thread. 5000 × 1s = no trading day left.
Serial indicator calculation loops	50 – 100ms	**~4–8 hrs**	Pure Python for-loops. No vectorisation. CPU sits at 5% utilisation.
SQLite read/write per signal	5 – 20ms	**~15 min**	Disk I/O on every symbol destroys throughput.
3-stage funnel (correct approach)	< 2ms avg	**< 10 sec**	Bulk fetch → vectorised filter → parallel deep scan on survivors only.

The 3-Stage Funnel — Architecture

Think of it as a funnel. Pour in 5000 stocks at the top. Each stage is cheap and fast, eliminating the majority. Only the ~50 best candidates reach the expensive Stage 3.

Stage 1 — Bulk Bhav Copy Filter (0.5 sec | 5000 stocks → ~200)

NSE publishes a single CSV file every day with OHLCV for ALL listed stocks. One HTTP request. No individual symbol calls. Download it into pandas once — filter everything in vectorised operations.

- URL: https://nsearchives.nseindia.com/content/cm/BhavCopy_NSE_CM_0_0_0_{DDMMYYYY}_F_0000.csv
- Load: df = pd.read_csv(url) → instant 5000-row DataFrame, ~500 KB file
- Apply ALL filters as vectorised boolean masks — no Python loops:
  - Volume: df = df[df['TOTTRDQTY'] > 500_000] — eliminates ~60% of stocks instantly (illiquid)
  - Price: df = df[df['CLOSE'].between(20, 15_000)] — removes penny stocks + ultra-mega cap outliers
  - Price move: df = df[df['CLOSE'].div(df['PREVCLOSE']).sub(1).abs() > 0.003] — stock must have moved at least 0.3% today
  - Not at circuit limit: exclude stocks exactly at 5%/10%/20% upper or lower circuit
  - Delivery %: if delivery data available, df = df[df['DELIV_PER'] > 25] — filters out pure speculators
- Result: ~200 candidates in 0.5s. Zero per-symbol API calls made.

Tip: Also maintain a hard watchlist (NIFTY 500 symbols) — always include these even if they fail Stage 1 volume filter, since large-caps sometimes have low intraday volume but are still tradeable.

Stage 2 — Parallel Cache-First Fetch (3–5 sec | 200 stocks → ~50)

Now fetch 60-day OHLCV for the ~200 survivors. The trick: check the Parquet cache first. On a normal trading day, 80%+ of these stocks were cached at market open — only the incremental candle needs updating.

- Cache hit path (fast): load from data/cache/ohlcv/SYMBOL_1d.parquet. Takes < 1ms. Zero network.
- Cache miss path (parallel): ThreadPoolExecutor(max_workers=50) fires all 200 requests at once
- 50 concurrent workers × avg 0.4s response = all 200 stocks fetched in ~1.6s total
- After fetching, apply quality pre-filters (vectorised numpy per symbol, < 0.1ms each):
  - Average daily volume over 20 days > 200,000 shares
  - ATR(14) / Close > 0.008 — stock has enough range to profit after slippage
  - Not in a flat channel: require at least 3% price range over last 10 days
  - Trend filter: 20-day EMA must not be within 0.2% of 50-day EMA (sideways chop excluded)
- Result: ~50 high-quality candidates, all data in memory, ready for Stage 3

Stage 3 — Full TITAN Analysis (2–3 sec | 50 stocks → top 10 signals)

Only now run the full multi-strategy analysis. On 50 stocks with vectorised indicators, this is trivially fast.

- Pre-compute ALL indicators for all 50 stocks using IndicatorEngine.compute_batch() — one call, parallel numpy
- Run all 45 TITAN strategies on pre-computed indicator arrays (no recalculation)
- Use multiprocessing.Pool(cpu_count()) — 4 CPU cores = 12 stocks each, all running simultaneously
- Score each stock: sigma_score = weighted sum of strategy signals + volume confirmation + MTF alignment
- Sort by score descending → return top 10–15 candidates to CHIEF for capital allocation
- Entire Stage 3 including multiprocessing overhead: ~2s on 50 stocks, all 45 strategies

Pipeline total: Stage 1 (0.5s) + Stage 2 (3–4s) + Stage 3 (2s) = 5.5 – 6.5 seconds for all 5000 NSE stocks.

Intraday Speed — How to Stay Fast During Market Hours

The pre-open full scan runs once at 8:45 AM. During market hours (9:15–15:30), the system does NOT rescan all 5000 stocks every 15 minutes. Instead it uses incremental refresh on the shortlist.

- Intraday watchlist: the ~50 candidates from the morning scan + any news-triggered additions
- Every 15 min: fetch only the LATEST candle for these ~50 stocks (not full 60-day history)
- Each incremental fetch = 1 API call per stock, handled in parallel = ~1s for 50 stocks
- Re-run Stage 3 indicators on the updated arrays using numpy array slicing — no full recompute
- If a GLOBAL_EVENT fires (war, Fed decision, etc.): trigger targeted sector re-scan immediately. E.g. oil war → re-scan energy sector only (~30 stocks). Done in < 2s.
- At 14:45 PM (45 min before close): full post-market scan setup begins — identify overnight/swing candidates

The IndicatorEngine — Single Source of Truth

The biggest source of both slowness and code bloat in the current system is that each agent computes its own indicators independently. TITAN computes RSI. SIGMA computes RSI. KARMA computes RSI. Three copies of the same calculation per symbol per cycle.

Indicator	Python Loop Time	NumPy Vector Time	Speedup + Formula
EMA(20), EMA(50)	~5ms/stock	**~0.03ms/stock**	160× — pd.Series.ewm(span=n, adjust=False).mean()
RSI(14)	~8ms/stock	**~0.05ms/stock**	160× — np.clip(delta,0,None).rolling(n).mean() / same for loss
ATR(14)	~10ms/stock	**~0.06ms/stock**	165× — pd.concat([H-L, abs(H-Cp), abs(L-Cp)],axis=1).max(axis=1).rolling(n).mean()
VWAP	~3ms/stock	**~0.02ms/stock**	150× — (close*vol).cumsum() / vol.cumsum()
Bollinger Bands	~6ms/stock	**~0.04ms/stock**	150× — rolling(n).mean() ± 2×rolling(n).std()
MACD	~7ms/stock	**~0.04ms/stock**	175× — EMA(12) - EMA(26), signal = EMA(9) of MACD
ADX	~15ms/stock	**~0.09ms/stock**	165× — smoothed ratio of directional movement (+DI/-DI)
All 12 indicators	~80ms/stock	**~0.5ms/stock**	160× — 50 stocks: 4,000ms → 25ms total. 99.4% time saved.

Architecture: IndicatorEngine.compute_batch(symbols, df_dict) takes a dict of {symbol: DataFrame} and returns a dict of {symbol: {ema20, ema50, rsi, atr, macd, vwap, bb_upper, bb_lower, adx, obv, ...}}. Every agent reads from this shared result. Zero redundant computation.

Optional Speed Booster: Numba JIT

For the handful of indicators that cannot be easily vectorised with numpy (e.g. custom multi-bar pattern detection, walk-forward optimization loops), use Numba — a Python JIT compiler that compiles to native machine code.

- Add @numba.jit(nopython=True) above any function that loops over arrays
- First call compiles to native code (~0.5s). Every subsequent call runs at C speed.
- Typical speedup: 50–500× over pure Python loops. Comparable to numpy for most indicator math.
- Best use cases: custom candle pattern matching, walk-forward loop, KARMA weight update inner loop
- Install: pip install numba (requires LLVM — already available on most systems)
- Important: Numba works on numpy arrays, not pandas DataFrames. Convert with df.values before passing.

Scanning Modes — When to Run What

Mode	Time	Universe	Method	Purpose & Output
Pre-open	8:45–9:14 AM	All 5000	Bhav Copy + 3-stage funnel	Build today's watchlist. Output: top 50 candidates ranked by sigma score.
Opening scan	9:15–9:30 AM	Top 200	Parallel fetch + Stage 3	Catch opening gaps and first-candle breakouts. High-urgency signals.
Intraday	Every 15 min	Top 50	Incremental candle refresh	Update signals on watchlist only. Fast: ~1s per cycle.
News-triggered	On event	Affected sector	Targeted sector scan	React to global/macro events. E.g. oil war → energy sector only.
Pre-close	14:45 PM	Top 100	Full pipeline	Identify positional/swing candidates for next day.
Post-market	16:00 PM	NIFTY 500	Full pipeline + backtest	Deep scan + walk-forward on shortlist. Update KARMA weights.
Off-hours	19:00 PM	Top 200	Historical + walk-forward	Validate strategy performance. Prepare next day watchlist.

Files to Build

File	Key Class / Function	What It Does
src/scanner/universe.py	UniverseManager	Maintains NSE stock lists: NIFTY50, NIFTY500, ALL_NSE. Weekly refresh from NSE indices CSV. Cached to data/cache/universe.json.
src/scanner/bulk_filter.py	BulkFilter.run()	Downloads Bhav Copy (1 HTTP req), applies vectorised pandas filters, returns ~200 symbol list in < 1s.
src/scanner/parallel_fetch.py	ParallelFetcher.fetch()	ThreadPoolExecutor(50 workers), cache-aware (Parquet first), fetches 200 stocks OHLCV in parallel in ~3–5s.
src/scanner/indicator_engine.py	IndicatorEngine.compute_batch()	Takes {symbol: DataFrame} dict, returns {symbol: {all_indicators}} using pure numpy. Single shared computation for all agents.
src/scanner/screener.py	StockScreener.scan()	Orchestrator: Stage 1 → 2 → 3. Called by main.py. Returns ranked signal list to SIGMA/CHIEF. Replaces per-agent fetching.
src/scanner/numba_indicators.py	@jit decorated functions	Numba-compiled versions of: candle pattern detection, walk-forward inner loop, custom multi-bar setups. 50–500× speedup.

Section 7 — Code Optimisation: Fewer Lines, Faster Execution

Once the system runs stably in paper mode, the next phase is a structured refactor. Goal: cut ~40% of lines of code and achieve 5–10× faster per-iteration execution. These two goals are the same thing — bloated code is always slow code. Every duplicated function is extra CPU cycles.

Rule Zero — Profile Before Touching Anything

80% of execution time lives in 20% of the code. Never guess. Measure first, then fix only what the profiler finds.

- Command: python -m cProfile -o profile.bin main.py then python -m pstats profile.bin
- Or live profiling without modifying code: pip install py-spy → py-spy top --pid <PID>
- Or line-by-line: pip install line_profiler → add @profile decorator to slow function, run kernprof -l main.py
- What to look for: any function appearing > 1000 calls, or any single call > 100ms
- In trading systems, the hot spots are almost always: indicator calculation loops, JSON serialisation in tight loops, synchronous HTTP requests, SQLite writes inside per-symbol loops
- Fix ONLY the top 3 hot spots first. Measure again. Repeat.

The 7 Optimisation Strategies

1. Vectorise Every Indicator Loop (100–200× speedup — biggest win)

Any for-loop over price data in any agent file is a bug, not just a style issue. Replace with numpy/pandas array operations.

Indicator	WRONG — Python loop (slow)	RIGHT — NumPy vectorised (fast)
EMA	for i in range(1,n): ema[i] = alpha*c[i] + (1-alpha)*ema[i-1]	pd.Series(close).ewm(span=period, adjust=False).mean()
RSI	for i in range(n,len): avg_gain += ...; avg_loss += ... (loop)	delta=close.diff(); gain=delta.clip(lower=0).rolling(n).mean()
ATR	for i in range(n,len): tr = max(H-L, abs(H-prev_C), abs(L-prev_C))	pd.concat([H-L,(H-C.shift()).abs(),(L-C.shift()).abs()],axis=1).max(axis=1).rolling(n).mean()
VWAP	for i in range(len): vwap += price[i]*vol[i] / sum(vol[:i+1])	(close * volume).cumsum() / volume.cumsum()
SMA	for i in range(n,len): sma[i] = sum(close[i-n:i]) / n	pd.Series(close).rolling(window=n).mean()

- Rule: grep -rn 'for i in range(len' src/ — every result is a vectorisation candidate
- For loops that truly cannot be vectorised: use @numba.jit(nopython=True) — compiles to native machine code
- Numba rule: only works on numpy arrays, not pandas. Convert: arr = df['close'].values

2. Async I/O for All Network Calls (20–50× speedup on data fetching)

- Every requests.get() inside any agent is blocking the entire thread. One slow broker API = whole system stalls.

- Replace with aiohttp + asyncio.gather(): responses = await asyncio.gather(*[session.get(url) for url in urls])
- All 50 stocks fetched simultaneously instead of one by one. 50 × 0.5s serial → 0.5s parallel.
- Already partially in ParallelFetcher — extend the same pattern to HERMES, ORACLE, HERMES_GLOBAL
- For Telegram alerts: use python-telegram-bot async API — don't block the main loop on message delivery
- Use aiohttp.ClientSession() with a connection pool (connector=aiohttp.TCPConnector(limit=100)) — reuse connections

3. Shared IndicatorEngine — Remove 70% of CPU Redundancy

- Current problem: TITAN, SIGMA, KARMA, NEXUS, LENS all compute RSI/EMA/ATR independently every cycle
- For 16 agents × 50 stocks × 12 indicators = 9,600 individual calculations per cycle
- With a shared engine: 50 stocks × 12 indicators = 600 calculations total — 16× fewer CPU operations
- Pattern: at start of each cycle, main.py calls indicators = IndicatorEngine.compute_batch(data)
- Pass indicators dict to every agent via cfg['indicators']. Agents read from it, never recompute.
- Result: indicator calculation drops from ~4,000ms to ~25ms per cycle

4. DRY Refactor — Cut Lines Without Cutting Features

Pattern to Remove	Lines Saved (est.)	How to Consolidate	Where to Put It
Copy-pasted EMA/RSI/ATR in each agent	**~800 lines**	Delete all, import from IndicatorEngine	src/scanner/indicator_engine.py
Repeated try/except + logger.error pattern	**~400 lines**	Create @safe_agent_run decorator	src/utils/decorators.py
Repeated logging.getLogger + handler setup	**~200 lines**	One setup_logger(name) utility function	src/utils/logger.py
Repeated os.getenv() scattered everywhere	**~150 lines**	All config reads through Settings singleton	config/settings.py (already exists)
Empty stub agents (run(): pass/return {})	**~300 lines**	Delete entire file if agent not implemented	Don't stub — remove
Repeated market hours check in each agent	**~100 lines**	One is_market_open() from market_data.py	src/data/market_data.py (already exists)
Redundant DataFrame copies (df2 = df.copy())	**~200 lines**	Use df.pipe() or inplace ops where safe	Per-agent refactor during review
# Comment saying what the function name says	**~300 lines**	Delete: # This function computes EMA above def ema():	Pure deletion — no replacement needed

5. Batch Database Writes (3–10× I/O speedup)

- Current problem: LENS writes to SQLite on every signal, potentially 100s of times per minute
- Fix: accumulate evaluations in a Python list (in-memory buffer), flush to DB every 60s or at cycle end
- Pattern: self._write_buffer.append(record) — if len(self._write_buffer) >= 100 or cycle_end: self._flush()
- Same for logs/status.json: write once per cycle, not once per signal
- SQLite tip: wrap batch inserts in a single transaction: conn.executemany(sql, records) inside with conn: block
- Impact: reduces disk I/O from ~100 writes/min to 1 write/min — extends SSD life, eliminates I/O wait

6. Lazy Imports — Fix 15–30 Second Startup

- tensorflow, xgboost, transformers each take 2–8 seconds to import at module level
- Fix: only import them inside the function that needs them, guarded by a flag:
  - pattern: _xgb = None then def get_xgb(): global _xgb; if _xgb is None: import xgboost as xgb; _xgb = xgb; return _xgb
- Or use importlib.import_module() with a try/except to handle optional packages gracefully
- Target: startup from python main.py to 'All agents ready' in < 5 seconds
- Also remove any import * statements — they slow Python's namespace resolution and hide what's actually used

7. Memory Optimisation — From ~800 MB to < 200 MB

- Current problem: each agent holds its own copy of full OHLCV DataFrames for all symbols in memory
- Fix: central data store in main.py. Agents receive symbol names + slice references, not full copies
- Use numpy float32 instead of float64 for price data — halves memory for indicator arrays
- Limit historical depth in memory: agents only need last 200 candles for indicators. Don't hold 2 years.
- Delete DataFrames after use: del df; gc.collect() — Python doesn't free memory automatically in loops
- Use slots in dataclasses: @dataclass class Signal: __slots__ = ['symbol','action',...] — 40% less memory per object
- Profile memory: pip install tracemalloc → snapshot1 = tracemalloc.take_snapshot() to find leaks

The @safe_agent_run Decorator — Remove 400 Lines of Boilerplate

Every agent currently has nearly identical try/except/log/return blocks. Replace all of them with one decorator in src/utils/decorators.py:

BEFORE — Copy-Pasted in Every Agent (bloated)	AFTER — One Decorator Replaces All of This

`def run(self):`
`  try:`
`  result = self._do_work()`
`  return result`
`  except Exception as e:`
`  logger.error(f'Agent failed: {e}')`
`  self.status = 'error'`
`  return {}`
`  # Repeated 16 times across all agents`
	`# In src/utils/decorators.py (written ONCE):`
	`def safe_run(fn):`
	`  def wrapper(self, *a, **kw):`
	`  try: return fn(self, *a, **kw)`
	`  except Exception as e:`
	`  self.logger.error(str(e))`
	`  self.status = 'error'; return {}`
	`  return wrapper`
	
	`# In every agent — just ONE line:`

BEFORE — Copy-Pasted in Every Agent (bloated)	AFTER — One Decorator Replaces All of This
	`@safe_run` `def run(self): return self._do_work()`

Line Count Reduction — Target Per Module

Module	Lines Now (est.)	Lines Target	% Saved	Main Cuts
src/agents/titan_agent.py	~1,200	~600	**50%**	Remove per-strategy indicator loops → use IndicatorEngine
src/agents/sigma_agent.py	~800	~400	**50%**	Remove indicator code, use shared engine + @safe_run
src/agents/karma_agent.py	~700	~400	**43%**	Remove RSI/EMA copies, vectorise update loop, @safe_run
src/agents/hermes*.py	~600	~350	**42%**	Merge HERMES + HERMES_GLOBAL shared RSS logic into base class
src/data/market_data.py	~900	~500	**44%**	Remove per-provider duplicate error handling, use @retry decorator
src/agents/ (all 16 agents)	~7,000	~4,000	**43%**	Safe_run decorator + shared indicators + remove empty stubs
src/backtest/ + src/risk/	~1,500	~900	**40%**	Consolidate shared stat functions (sharpe, drawdown) into src/utils/stats.py
config/ + main.py	~800	~500	**37%**	Remove redundant .env reads, consolidate Settings, remove dead flags
TOTAL CODEBASE	~12,000	**~7,000**	**42%**	DRY refactor + shared engine + remove stubs + delete redundant comments

Optimisation Roadmap — Exact Order

- Step 1: Profile. Run cProfile for one full market-hours iteration. Print top 20 slowest functions. Write them down. Do not touch anything else.
- Step 2: Build IndicatorEngine (src/scanner/indicator_engine.py). Pure numpy, all 12 indicators in one function. Takes 2–3 days. Test against existing outputs to verify correctness.
- Step 3: Replace indicator code in TITAN first (largest agent). Import from IndicatorEngine. Run side-by-side comparison — outputs must match. Then roll out to SIGMA, KARMA, NEXUS one by one.

- Step 4: Build @safe_run and @retry decorators in src/utils/. Replace all try/except blocks in agents. Run full test cycle. Saves ~400 lines immediately.
- Step 5: Build BulkFilter + ParallelFetcher (the 3-stage scanner). Test pre-market. Wire to main.py as the new universe source. Replaces per-agent fetching entirely.
- Step 6: Add async I/O to DataFetcher. Convert requests.get() to aiohttp. Wrap in asyncio.gather(). Test data integrity — same results, faster.
- Step 7: Add write buffer to LENS. Batch SQLite writes. Test that no evaluations are lost on crash (flush on SIGTERM).
- Step 8: Add lazy imports for tensorflow/xgboost/transformers. Measure startup time before/after.
- Step 9: Memory audit with tracemalloc. Find top 3 memory hogs. Fix.
- Step 10: Final code review — delete all empty stubs, dead code, redundant comments. Count lines.

Target Metrics After Full Optimisation

Metric	Before	After	Key Change
Startup time	15–30 seconds	**< 5 seconds**	Lazy imports — tensorflow/xgboost not loaded until needed
Per-iteration cycle time	~60 seconds	**< 10 seconds**	Shared IndicatorEngine + async I/O + batch DB writes
Full 5000-stock scan	4–8 hours	**< 10 seconds**	3-stage funnel: Bhav Copy bulk + parallel fetch + vectorised
Intraday refresh (50 stocks)	~25 seconds	**< 2 seconds**	Incremental candle fetch + cached indicators
RAM usage	~800 MB	**< 200 MB**	Central data store, float32 arrays, no agent-level DF copies
Lines of code	~12,000	**~7,000**	DRY: shared engine, decorators, remove stubs, delete dead code
SQLite writes	~100/minute	**~1/minute**	Write buffer — batch insert every 60s or at cycle end
Indicator calculations	9,600/cycle	**600/cycle**	Shared IndicatorEngine — computed once, read by all agents

Golden rule: Make it work → make it right → make it fast. In that order. Do NOT optimise until the system has run stably in paper mode for 2+ weeks. Premature optimisation introduces subtle bugs at the worst possible time — right before you go live with real money.